

AI-Based Plant Disease Detection Using Deep Learning and Transfer Learning

SRISANTH.S - 24619151

VASANTHARAMAN.S - 24619134

VENKATARAMANAN.R - 24619156

BERLIN.B - 24619139

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

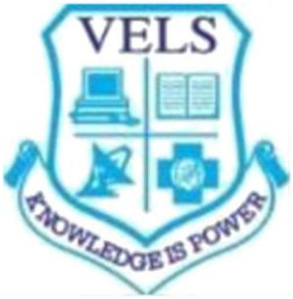
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"AI-BASED PLANT DISEASE DETECTION USING DEEP LEARNING AND TRANSFER LEARNING"** is the bonafide work carried out by **[SRISANTH (24619151)], [VASANTHARAMAN (24619134)], [VENKATARAMANAN (24619154)], and [BERLIN (24619139)]**, students of the Department of Artificial Intelligence and Machine Learning, Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, during the academic year **2024–2028**, under my guidance and supervision.

The project work submitted here is in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence and Machine Learning.

SIGNATURE OF THE HOD
Dr.K.Ulaga Priya M.E.,Ph.D
Associate Professor & Head
Of Department AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR
Dr.G.Sangeetha
Assistant Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the project work and viva-voice examination held on …… at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support I received from the positive minds around me in making this venture a successful one. The infrastructural support with all kinds of lab facilities have been a motivating factor in this completion of project work, all because of FOUNDER & CHAIRMAN, Shri. Dr. ISHARIK. GANESH., with great pleasure that I take this opportunity to thank him. I am grateful to our **HEAD OF THE DEPARTMENT, Dr.K.Ulaga Priya,** for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day. I extend our warmest thanks to my **INTERNAL GUIDE, Dr.G.Sangeetha,** who has patiently guided and provided us with invaluable advice and help. I would like to express my greatest gratitude to our Staff members, Librarian, and Non-teaching staff members of Computer Centre, who have helped and supported me throughout. Last but not the Least, I wish to thank my parents and my friends for their individual support and interest who has inspired me and encouraged me to go in my own way, without them I would be unable to complete this project.

SRISANTH.S - 24619151

VASANTHARAMAN.S - 24619134

VENKATARAMANAN.R - 24619156

BERLIN.B - 24619139

CHAPTER NO.	TABLE OF CONTENTS	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**ii**
	LIST OF TABLES	**iii**
	ABBREVIATIONS	**1**
1	1.Introduction	1
	1.1 Overview of the Project	1
	1.2 problem Statement	2
	1.3 Objectives of the Project	2
	1.4 Scope of the Project	3
	1.5 Significance of the Project	
2	**LITERATURE REVIEW**	**4**
	2.1 Introduction to Plant Disease	4
	2.2 Traditional Disease identification	4
	2.3 Machine Learning Approaches	4
	2.4 Deep Learning Approaches	5
	2.5 Transfer Learning and MobileNetV2	5
	2.6 Summary of Literature Review	
3	**SYSTEM DESIGN**	**7**
	3.1 Existing System	7
	3.2 Proposed System	7
	3.3 System Architecture	8
	3.4 Dataset Description	8
	3.5 Data preprocessing	8
	3.6 Hardware Requirement	8
	3.7 Software Requirements	9
4	Implementation and Testing	9
	4.1 Project Methodology	9

CHAPTER NO.	TABLE OF CONTENTS	PAGE NO
	4.2 Dataset Preparation	9
	4.3 Model Development	9
	4.4 Transfer Learning Using MobileNetV2	10
	4.5 Model Training	11
	4.6 Model Evaluation	11
	4.7 Flask Web Application	12
	4.8 Image Upload and Prediction	
	4.9 Testing	
5	**Results and Discussion**	**13**
	5.1 System Output	13
	5.2 Performance Discussion	13
	5.3 Limitations	
6	**Conclusion and Future Work**	14
	6.1 Conclusion	14
	6.2 Future Enhancements	15
	Import Source Code	15
	Reference	18
	Appendix and additional Screenshots	22

ABST RACT

Agriculture plays an important role in food production and economic development. However, plant diseases can reduce crop quality, affect agricultural productivity, and cause financial losses to farmers. Traditional plant disease identification generally depends on visual inspection and expert knowledge. This process can be time-consuming, particularly when a large number of plants need to be examined.

This project presents an **AI-based Plant Disease Detection System** that uses deep learning and transfer learning to classify plant leaf images into different disease and healthy categories. The system uses the PlantVillage dataset containing images of healthy and diseased leaves from multiple plant species. The images are preprocessed and resized to 224 × 224 pixels before being used for model training.

A MobileNetV2-based deep learning model is developed using TensorFlow and Keras. Transfer learning is used to take advantage of features learned from a previously trained model and adapt them to plant disease classification. Data augmentation techniques are applied during training to introduce image variations and help the model generalise to different leaf appearances.

The trained model is integrated into a web application developed using Python and Flask. Users can upload a plant leaf image through the web interface, and the application processes the image and displays the predicted class along with its confidence score.

The developed prototype demonstrates how artificial intelligence can assist in preliminary plant disease identification through image classification. During the reported training process, the model achieved **92.92% training accuracy and 94.11% validation accuracy**. These values represent training and validation performance and should not be interpreted as independent test accuracy.

The proposed system provides a foundation for accessible, image-based plant disease screening. However, predictions may be affected by image quality, lighting, background conditions, and differences between dataset images and real-world plant photographs. Therefore, the system should be considered a decision-support tool rather than a replacement for professional agricultural diagnosis.

Keywords: Artificial Intelligence, Plant Disease Detection, Deep Learning, Transfer Learning, MobileNetV2, TensorFlow, Keras, Flask, Image Classification.

CHAPTER 1

INTRODUCTION

1.1 Overview of the Project

Agriculture is one of the major sectors that supports human life by providing food, raw materials, and employment opportunities. Healthy crops are essential for maintaining agricultural productivity and ensuring the availability of quality food. However, plant diseases can negatively affect crop growth, reduce yield, damage the quality of agricultural products, and cause economic losses to farmers.

Plant diseases may develop due to fungal, bacterial, viral, and other biological factors. The symptoms of these diseases may appear as leaf spots, yellowing, discoloration, wilting, or damaged leaf surfaces. Identifying these symptoms at an early stage can help farmers take appropriate action and reduce the potential spread of diseases.

Traditionally, plant diseases are identified through visual inspection by farmers or agricultural experts. Although this approach is useful, it can require considerable time and experience. In addition, some diseases have similar visual symptoms, making accurate identification difficult for inexperienced users.

Artificial Intelligence (AI) provides opportunities to automate image analysis and classification. Deep learning, particularly Convolutional Neural Networks (CNNs), has been widely applied to image recognition tasks because it can learn useful visual features directly from image data. These capabilities make deep learning a suitable approach for developing automated plant leaf classification systems.

The project titled **"AI-Based Plant Disease Detection Using Deep Learning and Transfer Learning"** aims to develop a web-based application that classifies plant leaf images into predefined healthy and diseased categories. The application uses a MobileNetV2-based model developed with TensorFlow and Keras. Transfer learning is used to adapt a pretrained image recognition model to the plant disease classification task.

The system uses the PlantVillage dataset for model development. The images are preprocessed and resized to a standard input dimension of 224 × 224 pixels. The model is trained to identify visual patterns associated with the classes represented in the dataset.

A web interface is developed using Python and Flask. Users can open the application, upload a plant leaf image, and receive the model's predicted class and confidence score. This provides a simple way to demonstrate image-based plant disease classification without requiring users to interact directly with the

machine learning code.

The project combines dataset preparation, image preprocessing, transfer learning, model training, prediction, and web application development. It demonstrates how AI techniques can be integrated into a practical software prototype to support preliminary plant disease identification.

1.2 Problem Statement

Plant disease identification through manual inspection can be time-consuming and may require specialised agricultural knowledge. Farmers and other users may find it difficult to distinguish between diseases that produce similar symptoms on plant leaves. Delayed or incorrect identification can affect decisions related to crop management.

There is therefore a need for a computer-based system that can analyse plant leaf images and classify them into predefined disease or healthy categories. Such a system should provide a simple user interface, process uploaded images, and display the predicted result in an understandable format.

The problem addressed by this project is the development of a deep learning-based web application that uses plant leaf images to predict the corresponding class from a predefined set of categories.

The system is designed to assist preliminary identification. Its prediction depends on the quality and characteristics of the input image and the classes learned during training. It does not replace laboratory testing or expert agricultural diagnosis.

1.3 Objectives of the Project

The main objective of this project is to develop an AI-based plant disease detection application using deep learning and transfer learning.

The specific objectives are:

1. To use a labelled dataset containing healthy and diseased plant leaf images.
2. To prepare and preprocess images for deep learning model training.
3. To develop a plant disease classification model using MobileNetV2 and TensorFlow/Keras.
4. To apply transfer learning to reuse visual features learned by a pretrained model.
5. To train the model to classify images into the supported plant disease and healthy categories.
6. To save the trained model and the corresponding class names for later prediction.

7. To develop a web interface using Python and Flask.
8. To allow users to upload plant leaf images through the web application.
9. To display the predicted class and confidence score for the uploaded image.
10. To examine the model's training and validation performance and identify its limitations.

1.4 Scope of the Project

The scope of the project includes the development of a prototype application for image-based plant disease classification.

The system uses a labelled plant leaf image dataset containing multiple healthy and diseased categories. The dataset is organised into class folders so that the images can be associated with their corresponding labels during model training.

Image preprocessing is performed to convert input images into a standard size suitable for the selected model. Data augmentation is applied during training to introduce variations in the training images. These variations can help reduce overdependence on particular image orientations or appearances.

MobileNetV2 is used as the feature extraction foundation for the classification model. The model is developed with TensorFlow and Keras, trained using the prepared dataset, and saved for use by the prediction application.

The web application is developed using Flask. It provides an interface through which a user can upload an image and view the predicted category and confidence score.

The current project focuses on classifying images into the categories represented in its training dataset. It does not independently verify the biological cause of a disease, prescribe treatments, or guarantee accurate predictions for every plant photograph. Its performance may differ when images are captured in real agricultural environments rather than under dataset conditions.

1.5 Significance of the Project

The project demonstrates the practical application of artificial intelligence in agriculture. By combining image classification with a web interface, it provides an accessible way to explore automated plant disease identification.

The system can reduce the effort involved in manually examining every image during preliminary

screening. It can also demonstrate how transfer learning can be used to develop an image classification model without designing an entire deep neural network from the beginning.

The project has educational value because it integrates several important areas of computer science, including Python programming, dataset preparation, image processing, deep learning, model evaluation, and web application development.

The system may also serve as a starting point for future agricultural applications. With additional testing on diverse real-world images and evaluation by agricultural experts, similar systems could be further developed to support crop monitoring and disease screening.

However, the system's usefulness depends on the reliability of its predictions. Users should verify uncertain or important results with qualified agricultural professionals before making decisions about crop treatment.

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolution Neural Network
TL	Transfer Learning
API	Application Programming Interface
UI	User Interface
GUI	Graphical User Interface
HTML	Hypertext Markup Language
CSS	Cascading Style sheets
HTTP	Hypertext Transfer Protocol
CPU	Central Processing Unit
ReLU	Rectified Linear Unit

CHAPTER 2

LITERATURE REVIEW

2.1 Introductions

Plant disease detection has become an important research area in agriculture because plant diseases can affect crop growth, productivity, and quality. Early identification of diseases helps farmers take appropriate action to manage affected plants. Various techniques have been developed to identify plant diseases, including visual inspection, image processing, machine learning, and deep learning.

Traditional methods mainly depend on human observation and agricultural expertise. Recent developments in artificial intelligence have introduced automated methods that analyse plant leaf images and classify them into different categories.

Deep learning techniques, particularly Convolutional Neural Networks (CNNs), have shown potential for image classification because they can learn useful visual features from training images. Transfer learning further improves the development process by adapting pretrained models to specific classification tasks.

This chapter discusses traditional plant disease identification, machine learning approaches, deep learning techniques, transfer learning, and the MobileNetV2 architecture used in the proposed system

2.2 Traditional Plant Disease Identification

Traditional plant disease identification is generally performed through visual examination of leaves, stems, fruits, and other plant parts. Farmers and agricultural experts observe symptoms such as leaf spots, yellowing, wilting, discoloration, and unusual growth patterns.

This approach is simple and does not require specialised computer equipment. Experienced agricultural professionals may identify many diseases through visible symptoms and their knowledge of crop conditions.

However, traditional identification has several limitations. It may require considerable time when many plants need to be inspected. The accuracy of identification can depend on the observer's experience. Furthermore, different diseases may produce similar symptoms, making it difficult to distinguish them through visual inspection alone.

These limitations have encouraged researchers to explore computer-based techniques for analysing plant images and supporting preliminary disease identification.

2.3 Machine Learning Approaches

1. achine learning is a branch of artificial intelligence that enables computer systems to learn patterns from data and make predictions. In plant disease classification, machine learning algorithms can be trained using features extracted from plant leaf images.

2. A conventional machine learning workflow may include image acquisition, preprocessing, feature extraction, model training, and classification. Features may describe colour, texture, shape, and the distribution of visible spots on a leaf.

3. Algorithms such as Support Vector Machines, Decision Trees, and Random Forests can be used to classify images based on extracted features. Their performance depends on the quality of the features, the dataset, and the selected algorithm.

4. A major limitation of conventional approaches is that feature extraction may require manual design or specialised processing techniques. Important visual patterns may not be represented adequately if the selected features do not capture the characteristics of a disease.

5. Deep learning provides an alternative by learning image features automatically during training.

2.4 Deep Learning for Plant Disease Detection

Deep learning uses neural networks containing multiple processing layers to learn complex patterns from data. For image classification, Convolutional Neural Networks are commonly used because they can learn spatial features from images.

A CNN typically contains convolutional layers, activation functions, pooling operations, and classification layers. Early layers can learn simple visual patterns such as edges and textures, while deeper layers can learn more complex patterns.

In plant disease classification, these learned features may help distinguish between healthy leaves and leaves showing disease symptoms. The model learns from labelled examples and uses the learned patterns to predict the class of an input image.

The quality of a deep learning model depends on several factors, including the size and diversity of the dataset, image quality, class distribution, model architecture, and training procedure.

Deep learning models may also face difficulties when the training images differ significantly from real-world images. Changes in lighting, background, camera quality, and leaf orientation can affect prediction

performance.

Therefore, appropriate preprocessing, dataset preparation, and model evaluation are important parts of a reliable plant disease classification system.

2.5 Transfer Learning

Transfer learning is a technique in which knowledge learned by a model on one task is reused for another related task. In image classification, a model pretrained on a large image dataset can provide useful visual features for a new classification problem.

Training a deep learning model entirely from the beginning may require substantial data and computational resources. Transfer learning can reduce the amount of training required by reusing features learned by an existing model.

In the proposed project, transfer learning is used with MobileNetV2. The pretrained network acts as the foundation for processing plant leaf images. A classification component is adapted to predict the plant disease categories represented in the dataset.

The resulting model is trained using the prepared plant leaf images. Its performance is assessed using training and validation data.

Transfer learning is useful for developing practical image classification applications, but it does not automatically guarantee high accuracy. The model must still be trained and evaluated carefully using appropriate data.

2.6 MobileNetV2 Architecture

MobileNetV2 is a convolutional neural network architecture designed to provide efficient image processing with relatively low computational requirements compared with many larger image classification models.

The architecture uses depthwise separable convolutions to reduce the number of computations and parameters required for feature extraction. It also uses inverted residual blocks and linear bottlenecks to support efficient feature processing.

These design characteristics make MobileNetV2 suitable for applications where computational efficiency is important.

In this project, MobileNetV2 is used as the basis of the plant disease classification

model. Input images are resized to 224 × 224 pixels before being processed by the model. The extracted features are used by the classification component to predict one of the supported plant disease or healthy categories.

The model is developed using TensorFlow and Keras. After training, the model is saved and loaded by the Flask application for image prediction.

2.7 PlantVillage Dataset

The PlantVillage dataset is a collection of labelled plant leaf images commonly used in research on automated plant disease classification. It contains images from multiple plant species and includes healthy and diseased categories.

In this project, the dataset is used to train and validate the classification model. Images are organised according to their corresponding class labels so that the model can learn the relationship between visual patterns and categories.

The images are resized to a standard dimension of 224 × 224 pixels. Data augmentation techniques are applied during training to introduce variations in the training samples.

The project is configured to work with 38 classes. These classes represent the healthy and diseased categories included in the selected dataset.

Although PlantVillage is useful for model development, its images may differ from photographs captured in actual agricultural environments. Therefore, performance on this dataset does not necessarily guarantee equivalent performance on all real-world plant images.

2.8 Summary of Literature Review

The literature review shows that plant disease identification has progressed from manual inspection and conventional image processing to machine learning and deep learning approaches.

Traditional methods can be effective when performed by experienced individuals, but they may require considerable time and specialised knowledge. Conventional machine learning approaches can automate classification but may depend on manually selected image features.

Deep learning models can learn useful visual features directly from images. Transfer learning provides a practical method for adapting pretrained networks to plant disease classification. MobileNetV2 offers an efficient architecture for building image classification applications.

Based on these concepts, the proposed project combines MobileNetV2, TensorFlow/Keras, the PlantVillage dataset, and a Flask-based web interface to create a prototype plant disease classification system.

The next chapter describes the proposed system architecture, dataset organisation, hardware requirements, software requirements, and overall system design.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

System design is an important stage in software development. It describes the structure of the system, its major components, the flow of data, and the interaction between different modules.

The proposed project, **AI-Based Plant Disease Detection Using Deep Learning and Transfer Learning**, is designed to classify plant leaf images using a trained deep learning model. The system combines Python, TensorFlow/Keras, MobileNetV2, and Flask to provide a web-based interface for image classification.

The system consists of two major processes: model development and image prediction. During model development, labelled plant leaf images are prepared and used to train the classification model. During prediction, a user uploads a leaf image through the web application. The application preprocesses the image and passes it to the trained model. The predicted class and confidence score are then displayed to the user.

This chapter explains the existing system, proposed system, system architecture, dataset description, preprocessing techniques, and hardware and software requirements.

3.2 Existing System

In the existing approach, plant diseases are commonly identified through manual observation of visible symptoms on leaves and other plant parts. Farmers or agricultural experts examine changes such as yellow spots, brown patches, leaf discolouration, and other unusual patterns.

Although manual inspection is useful, it has certain limitations. The identification process may take considerable time when a large number of plants must be examined. Correct identification can also depend on the experience and knowledge of the person performing the inspection.

Some computer-based approaches use traditional image processing and machine learning techniques. These methods may require manually selected features, such as colour, shape, and texture, to classify plant images.

3.2.1 Limitations of the Existing System

The major limitations are:

1. Manual identification can be time-consuming.
2. Disease identification may depend on the observer's experience.
3. Similar symptoms can make different diseases difficult to distinguish.
4. Conventional machine learning approaches may require manual feature extraction.
5. Manual inspection can be difficult to scale across large numbers of plants.
6. The reliability of image-based identification can be affected by image quality and environmental conditions.

These limitations motivate the development of an automated image classification system that can support preliminary plant disease identification.

3.3 Proposed System

The proposed system uses deep learning and transfer learning to classify plant leaf images into one of the supported healthy or diseased categories.

The system uses MobileNetV2 as the foundation of the image classification model. The model is developed using TensorFlow and Keras and trained using labelled images from the PlantVillage dataset.

A Flask web application provides the user interface. Users can upload plant leaf images through a web browser. The application receives the uploaded image, preprocesses it into the required input format, and passes it to the trained model for prediction.

The predicted class is identified using the class names stored in `class_names.json`. The application then displays the predicted category and confidence score on the result page.

3.3.1 Main Modules of the Proposed System

1. Dataset Preparation Module

This module organises the PlantVillage images into class-based folders. The dataset contains 38 supported classes representing healthy and diseased plant categories.

2. Image Preprocessing Module

The input images are resized to 224 × 224 pixels to match the model's expected input dimensions. The image values are prepared according to the preprocessing requirements of the model.

3. Model Training Module

The MobileNetV2-based model is trained using TensorFlow/Keras. Training data is used to learn visual patterns, while validation data is used to monitor model performance during development.

4. Model Storage Module

The trained model is saved as `plant_disease_model.keras`. The corresponding class names are stored in `class_names.json`. These files are used by the application during prediction.

5. Web Application Module

The Flask application provides the web pages through which users can upload images and view prediction results.

6. Prediction and Result Module

The uploaded image is processed by the trained model. The application maps the predicted output to its corresponding class name and displays the result with a confidence score.

3.3.2 Advantages of the Proposed System

1. Automates plant leaf image classification.
2. Uses transfer learning to reuse pretrained image features.
3. Provides a simple web interface for uploading images.
4. Displays the predicted category and confidence score.
5. Uses a saved model for prediction without retraining it for every uploaded image.
6. Combines deep learning with a lightweight web application framework.

The system is intended as a prototype for preliminary plant disease classification. Its predictions should not be treated as a guaranteed agricultural diagnosis

3.4 System Architecture

The system architecture describes the relationship between the dataset, preprocessing module, model training process, saved model, and web application.

The architecture is divided into two main workflows: model development and user prediction.

3.4.1 Model Development Workflow

The model development process begins with the PlantVillage dataset. The images are organised into their corresponding classes and prepared for training.

The images are resized to 224 × 224 pixels. Data augmentation techniques, such as rotation, horizontal flipping, and zoom, are applied during training to introduce variations into the training samples.

The prepared images are passed to the MobileNetV2-based model. Transfer learning is used to adapt the pretrained architecture to the plant disease classification task. The model is trained and evaluated using the configured training and validation data.

After training, the model is saved in Keras format. The class labels are saved separately in a JSON file. Both files are used by the Flask application during prediction.

3.4.2 User Prediction Workflow

The user prediction process begins when a user opens the Flask web application in a browser and selects a plant leaf image.

The application receives the uploaded image and prepares it for prediction. The image is resized to the required dimensions and processed using the same input preparation method expected by the trained model.

The trained model analyses the image and produces prediction scores for the supported classes. The application identifies the predicted class and obtains its corresponding name from `class_names.json`.

Finally, the result page displays the predicted plant category and the associated confidence score .

3.4.3 Architecture Flow

The overall architecture can be represented as follows:

Model Development:

PlantVillage Dataset
↓
Dataset Organisation
↓
Image Preprocessing and Augmentation
↓
MobileNetV2 Transfer Learning
↓
Model Training and Validation
↓
Save Trained Model and Class Names
↓
`plant_disease_model.keras` + `class_names.json`

User Prediction:

User
↓
Flask Web Interface
↓
Upload Plant Leaf Image
↓
Image Preprocessing
↓
Trained MobileNetV2-Based Model
↓
Predicted Class and Confidence Score
↓
Display Result on Web Page

Figure 3.1: System Architecture of AI-Based Plant Disease Detection

Insert the system architecture diagram after this section.

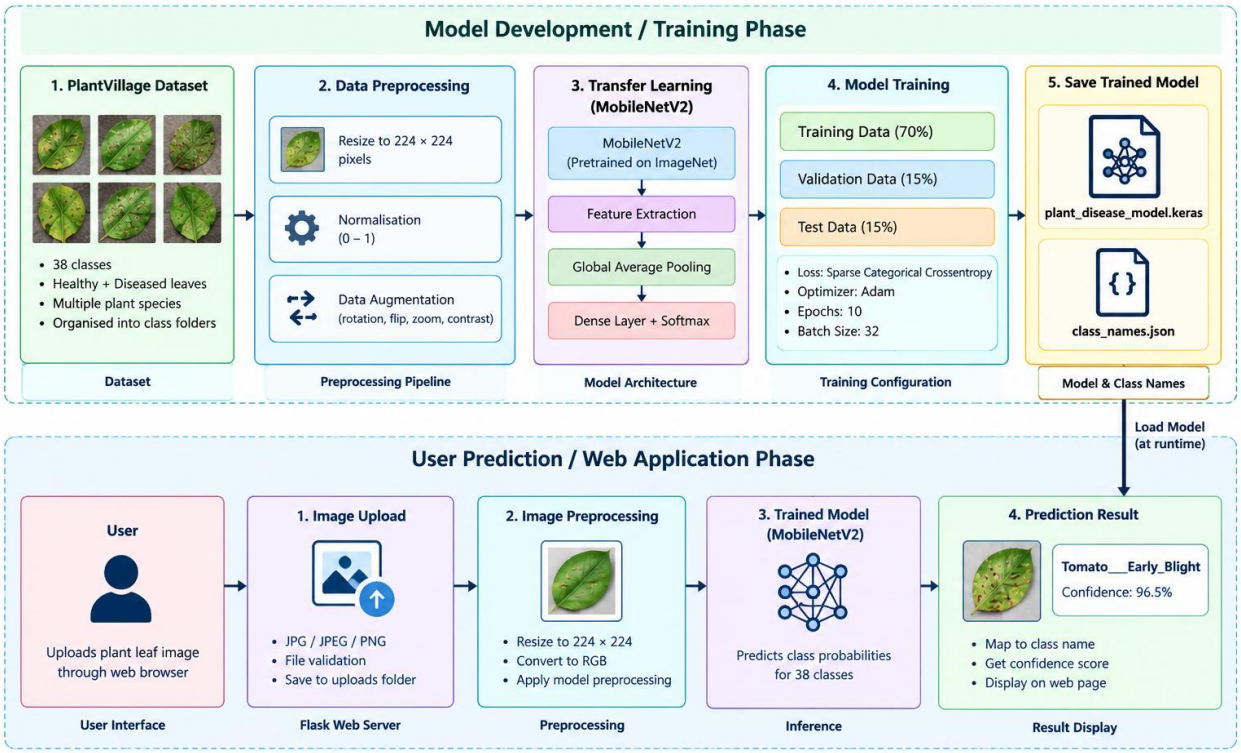


Figure 3.1 System Architecture of AI-Based Plant Disease Detection

Figure 3.1: System Architecture of Plant Disease Detection

3.5 Dataset Description

The PlantVillage dataset is used as the image source for training and validating the plant disease classification model. It contains labelled plant leaf images representing different plant species and disease conditions.

The selected project dataset is configured to support 38 classes. Each class represents a particular healthy or diseased category. The class-based organisation allows the model to learn the relationship between image patterns and their corresponding labels.

3.5.1 Dataset Characteristics

The main characteristics of the dataset used in the project are:

- **Dataset name:** PlantVillage
- **Number of classes:** 38
- **Input type:** Plant leaf images

- **Image format:** Supported digital image formats
- **Image size used by the model:** 224 × 224 pixels
- **Purpose:** Training and validation of the plant disease classification model
- **Model framework:** TensorFlow/Kreas

The dataset includes both healthy and diseased leaf categories. The model learns from the labelled images and uses the learned visual patterns to classify new images.

3.5.2 Dataset Organisation

The images are organised into class-based folders so that the training program can associate each image with its correct label.

A simplified example of the folder structure is shown below:

```
data/
├── train/
│   ├── Class_1/
│   ├── Class_2/
│   ├── Class_3/
│   └── ...
└── validation/
    ├── Class_1/
    ├── Class_2/
    ├── Class_3/
    └── ...
```

The folder names represent the actual class labels. The number of images in each folder depends on the selected dataset and its preparation process.

The exact folder structure used in the final report should match the structure used by the project's training code.

3.6 Data Preprocessing

Data preprocessing is performed to prepare the leaf images for input into the deep learning model. Consistent preprocessing is important because the model expects images of a specific size and input format.

3.6.1 Image Resizing

The original images may have different dimensions. Therefore, each image is resized to 224 × 224 pixels before being passed to the model.

Resizing provides a consistent input shape and allows the images to be processed in batches during training and prediction.

3.6.2 Image Normalisation and Model-Specific Preprocessing

Image pixel values must be prepared according to the input requirements of the model. Depending on the implementation, this may involve scaling pixel values or applying the preprocessing function associated with MobileNetV2.

The same compatible preprocessing method must be used during both training and prediction. Inconsistent preprocessing may reduce classification performance.

3.6.3 Data Augmentation

Data augmentation introduces variations into the training images. It can help the model learn patterns that are less dependent on the exact orientation or appearance of an individual image.

The project uses augmentation techniques such as:

- Horizontal flipping
- Image rotation
- Zoom variation
- Contrast variation

These transformations are applied during training according to the configured data pipeline. They are intended to improve the diversity of training examples.

Data augmentation does not replace the need for a diverse dataset or proper evaluation.

3.7 Hardware Requirements

The project is developed and tested as a local Python web application. The hardware required depends on the size of the dataset and the computational resources needed for model training.

The following table describes the general hardware requirements for the project.

Component	Requirement
Computer	Desktop computer or laptop
Processor	Modern multi-core processor
Ram	8 GB recommended for general development
Storage	Sufficient free space for the dataset, model files, and project files
Graphics processor	Optional; training may be performed using a supported GPU or CPU
Display	Standard monitor or laptop display
Network	Internet connection may be required to obtain dependencies or the dataset

These are general recommendations rather than measured specifications of the computer used for development. The actual hardware specifications should be added if required by the institution.

3.8 Software Requirements

The project uses Python-based machine learning tools and a web framework to develop the plant disease classification application.

Software or Technology	Purpose
Operating System	Windows
Programming Language	Python
Code Editor	Visual Studio Code
Deep Learning Framework	TensorFlow
Model API	Keras
Pretrained Architecture	MobileNetV2
Web Framework	Flask
Front-End Technologies	HTML,CSS and JavaScript, where used
Model File Format	Keras .keras format
Class Lable File	JSON
Web Browser	Any compatible modern browser

3.8.1 Python

Python is used to implement the model training process, image preprocessing, prediction logic, and web application. Its libraries support the development of machine learning and image classification applications.

3.8.2 TensorFlow and Keras

TensorFlow and Keras are used to construct, train, save, and load the deep learning model. They provide tools for transfer learning and image classification

3.8.3 MobileNetV2

MobileNetV2 is used as the pretrained neural network architecture for feature extraction. The classification model is adapted to predict the supported plant disease categories.

3.8.4 Flask

Flask is a Python web framework used to connect the trained model with the user interface. It handles web requests, receives uploaded images, runs prediction, and returns the result page.

3.8.5 HTML and CSS

HTML is used to structure the web pages, while CSS is used to control their appearance and layout. JavaScript may be used for client-side interactions where implemented.

3.9 Summary

This chapter described the design of the proposed AI-based plant disease detection system. The system combines a PlantVillage dataset, image preprocessing, MobileNetV2 transfer learning, and a Flask web application.

The model development workflow prepares and trains the image classifier, while the prediction workflow allows users to upload plant leaf images and view the predicted category and confidence score.

The hardware and software requirements were also discussed. The next chapter explains the implementation process, including dataset preparation, model training, model saving, Flask integration, and testing.

CHAPTER 4

MPLEMENTATION AND TESTING

4.1 Introduction

Implementation is the process of converting the proposed system design into a working application. In this project, an AI-based plant disease detection system is developed using Python, TensorFlow, Keras, MobileNetV2, and Flask.

The implementation involves preparing the dataset, preprocessing plant leaf images, developing the deep learning model, training the model, saving the trained model, and integrating it with a web application.

The Flask application allows users to upload plant leaf images through a web browser. The uploaded image is processed by the trained model, and the predicted plant category and confidence score are displayed on the result page.

This chapter explains the implementation steps, model development, web application development, and testing procedures.

4.2 Development Environment

The project is developed using Python as the primary programming language. Visual Studio Code is used as the development environment.

TensorFlow and Keras are used for deep learning model development, while Flask is used to create the web application. HTML and CSS are used to design the user interface.

The main technologies used are:

Technology	**Purpose**
Python	Main programming language
TensorFlow	Deep learning framework
Kreas	Model development and training
MobileNetV2	Pretrained model architecture
Flask	Web application development
HTML	Web page structure
CSS	Web page Styling

JavaScript	Client-side functionality, where implemented

4.3 Project Folder Structure

The project is organised into different folders and files to maintain a clear structure.

The major project files are:

```
AI-Disease-Detection/
│
├── app.py
├── train.py
├── predict.py
├── prepare_dataset.py
├── requirements.txt
│
├── model/
│   ├── plant_disease_model.keras
│   └── class_names.json
│
├── data/
│   ├── train/
│   └── validation/
│
├── templates/
│   ├── index.html
│   ├── scan.html
│   └── result.html
│
└── static/
    ├── style.css
    └── uploads/
```

The `app.py` file contains the Flask application logic. The `train.py` file is used for model training, while `predict.py` handles image prediction logic.

The `model` folder stores the trained model and class names. The `templates` folder contains the HTML pages, and the `static` folder stores styling files and uploaded images.

The exact folder structure may vary depending on the final project configuration.

4.4 Dataset Preparation

The PlantVillage dataset is used for developing the plant disease classification model. It contains labelled images of healthy and diseased plant leaves.

The selected dataset is configured to support 38 different classes.

The dataset preparation process includes:

1. Collecting the plant leaf images.
2. Organising images according to their class names.
3. Preparing separate training and validation folders.
4. Checking the image files.
5. Preparing the dataset for model training.

The class-based folder structure allows the training program to automatically associate each image with its corresponding category.

4.5 Image Preprocessing

Image preprocessing is performed before training the deep learning model. It ensures that all images are prepared in a consistent format.

The following preprocessing operations are used:

4.5.1 Image Resizing

All input images are resized to 224 × 224 pixels. This ensures that the images have the dimensions required by the MobileNetV2-based model.

4.5.2 Image Normalisation

The image pixel values are prepared according to the preprocessing requirements of the model. Consistent

preprocessing is important during both model training and prediction.

4.5.3 Data Augmentation

Data augmentation is applied to introduce variations into training images. The techniques include:

- Horizontal flipping
- Rotation
- Zoom
- Contrast variation

These operations increase the variety of training samples and help the model learn different visual patterns.

4.6 Model Development Using MobileNetV2

The deep learning model is developed using the MobileNetV2 architecture with transfer learning.

MobileNetV2 is a pretrained convolutional neural network designed for efficient image classification. It is used as the foundation for extracting visual features from plant leaf images.

The pretrained architecture is adapted to the plant disease classification task by adding a classification component for the 38 supported classes.

The main stages of model development are:

1. Loading the MobileNetV2 architecture.
2. Using the pretrained network for feature extraction.
3. Adding the classification layers.
4. Preparing the model for training.
5. Training the model using the prepared dataset.
6. Monitoring the training and validation performance.

The model learns visual patterns associated with healthy and diseased leaves and uses these patterns to classify new images.

4.7 Model Training

Model training is the process of teaching the neural network to identify patterns from labelled plant leaf images.

The prepared training dataset is supplied to the MobileNetV2-based model. During training, the model processes images in batches and updates its trainable parameters to improve classification performance.

The project uses the following training configuration:

Parameter	Value
Model Architecture	MobileNetV2
Input Image Size	224 × 224
Batch Size	32
Number of Epochs	10
Dataset	PlantVillage
Number of Classes	38
Framework	TensorFlow/Keras

An epoch represents one complete pass through the training dataset.

During training, validation data is used to monitor how well the model performs on images that are not used for updating its parameters.

After the training process is completed, the trained model is saved for later use in the web application.

4.8 Model Saving

After training, the model is saved in Keras format so that it can be loaded by the Flask application without repeating the training process.

The trained model is stored using the following filename:

```
plant_disease_model.keras
```

The class names are stored separately in:

```
class_names.json
```

The class names file maintains the relationship between the model's output positions and the corresponding plant disease categories.

Both files are stored inside the `model` directory.

4.9 Flask Web Application Development

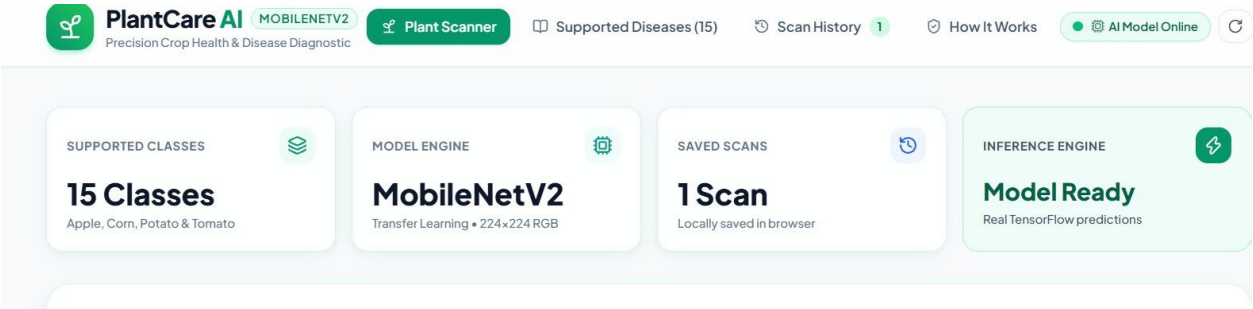
Flask is used to develop the web application and connect the trained deep learning model with the user interface.

The application provides a simple interface through which users can upload plant leaf images and view the prediction results.

The main Flask application is implemented in the `app.py` file.

4.9.1 Home Page

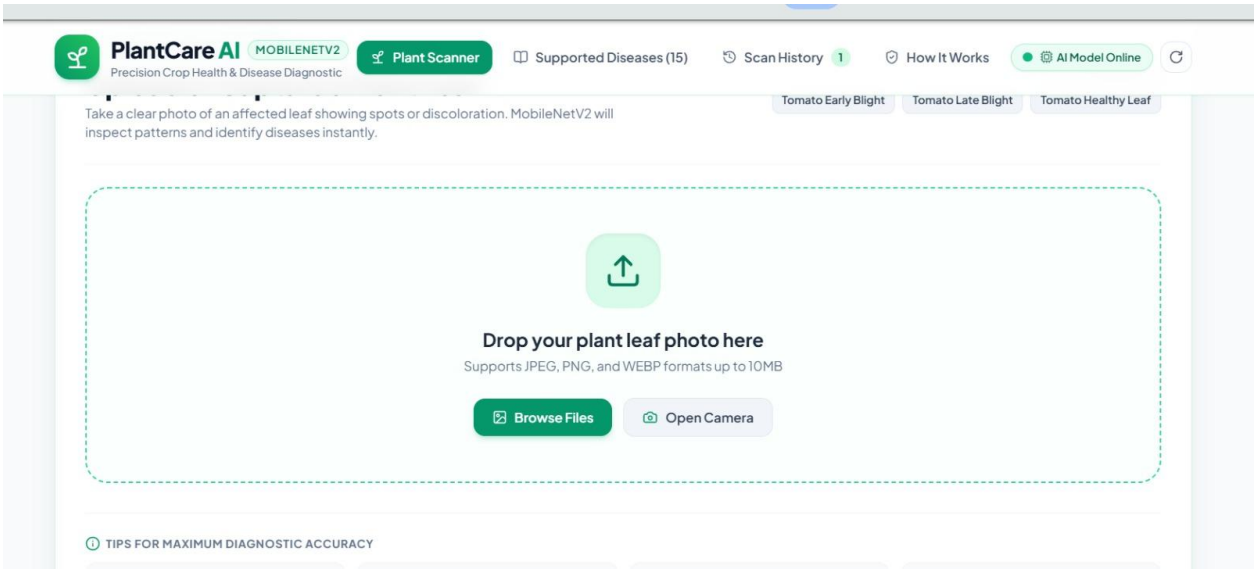
The home page provides the main interface of the application. It introduces the plant disease detection system and allows users to access the image scanning page.



4.9.2 Image Upload Page

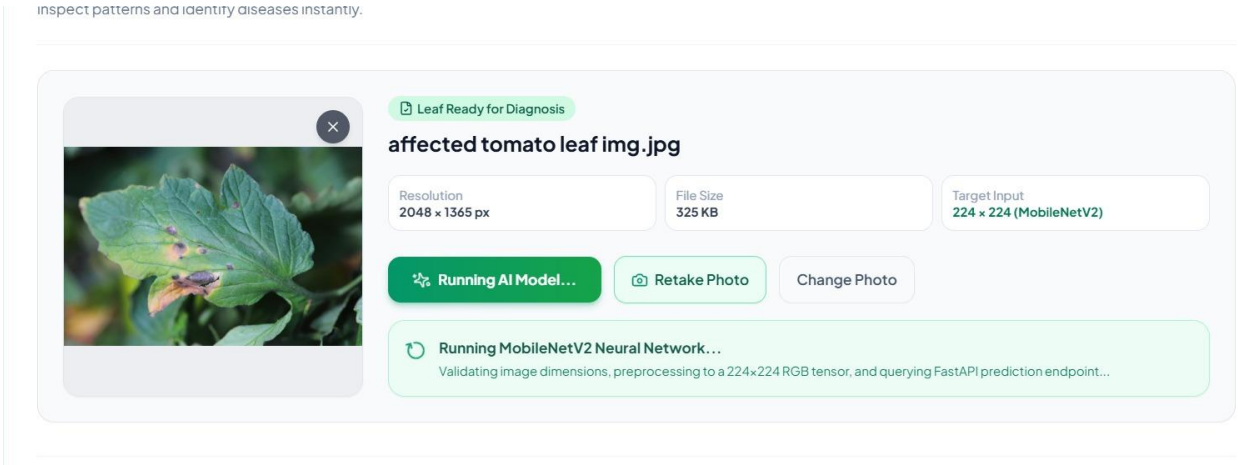
The scanning page allows users to select and upload a plant leaf image from their device.

The application receives the uploaded image and stores it in the configured upload directory.



4.9.3 Prediction Process

After receiving the uploaded image, the Flask application performs the following operations:



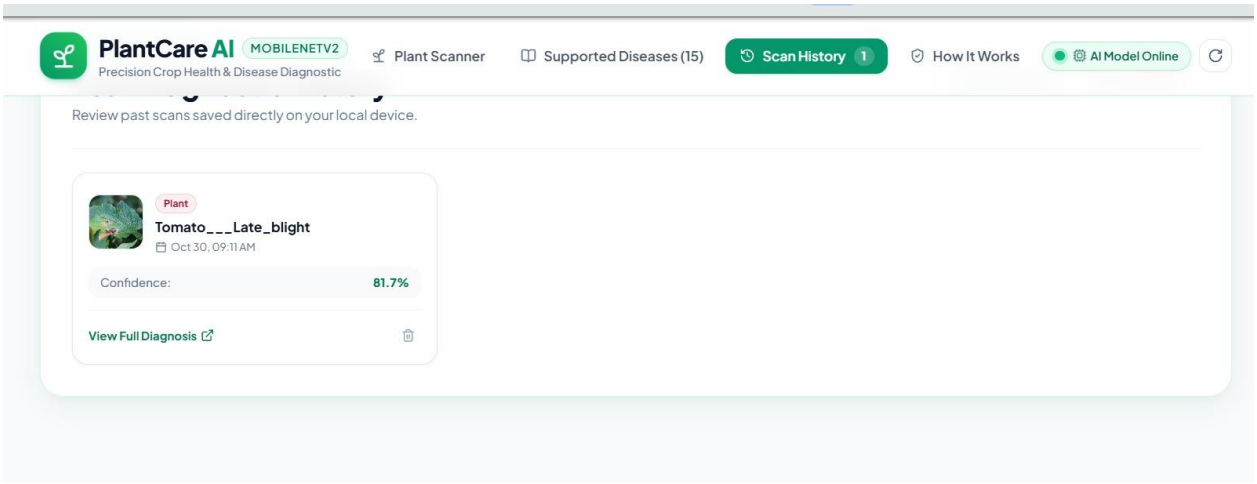
1. Receives the uploaded image.
2. Checks and processes the image.
3. Resizes the image to 224 × 224 pixels.
4. Applies the required model preprocessing.
5. Passes the image to the trained model.
6. Obtains prediction scores for the supported classes.

7. Identifies the predicted class using `class_names.json`.

4.9.4 Result Page

The result page displays the output generated by the trained model.

The displayed information includes:



- Uploaded plant leaf image.
- Predicted plant disease or healthy category.
- Confidence score of the prediction.

The result helps the user understand the model's classification of the uploaded image.

4.10 Working Procedure of the Application

The application follows a simple sequence of operations.

Step 1: Open the Application

The user starts the Flask application and opens it through a web browser using the local development address.

Step 2: Upload Image

The user navigates to the scanning page and selects a plant leaf image.

Step 3: Image Processing

The application receives the image and prepares it for the model by resizing and applying the required

preprocessing.

Step 4: Model Prediction

The processed image is passed to the trained MobileNetV2-based model.

Step 5: Result Generation

The model produces prediction scores. The application identifies the class with the highest predicted score.

Step 6: Display Result

The predicted category and confidence score are displayed on the result page.

4.11 Model Evaluation

Model evaluation is performed to understand the classification performance during development.

The project records the following training and validation accuracy values:

Evaluation Metric	Accuracy
Training Accuracy	92.92%
Validation Accuracy	94.11%

The training accuracy represents the model's performance on the training data. Validation accuracy represents its performance on the validation data.

These values indicate the model's performance during the reported training process. They should not be presented as independent test accuracy.

A separate test dataset evaluation is required to measure performance on unseen test images.

4.12 Software Testing

Software testing is performed to check whether the application components work as expected.

The testing process focuses on image uploading, image preprocessing, model prediction, and result

display.

4.12.1 Test Cases

The following table describes the main test cases for the application.

Test Case	Input / Action	Expected Result
Home Page	Open the application	Home page should load successfully
Scan Page	Open the scanning page	Open the scanning page
Valid Image Upload	Upload a support leaf image	Open the sscanningacnning page
Image preparation	Upload a support leaf image	Image should be prepared for the model

These are the functional test cases used to describe the intended application behaviour. Record the actual pass or fail status after executing each test.

4.13 Testing Results

The application is tested by uploading plant leaf images through the Flask web interface.

The uploaded images are processed and passed to the trained model. The model generates prediction scores, and the application displays the corresponding predicted category and confidence score.

The testing process checks the following functions:

1. Successful loading of the home page.
2. Navigation to the scanning page.
3. Uploading supported plant leaf images.
4. Processing uploaded images.
5. Generating model predictions.
6. Displaying prediction results.

The results of these tests help verify the integration between the trained model and the Flask web application.

4.14 Summary

This chapter explained the implementation and testing of the AI-based plant disease detection system.

The project uses Python, TensorFlow/Keras, and MobileNetV2 to develop the deep learning classification model. The PlantVillage dataset is used for model training and validation, and the trained model is saved for later use.

Flask is used to integrate the model with a web application. Users can upload plant leaf images and view the predicted category and confidence score.

The model achieved a training accuracy of 92.92% and a validation accuracy of 94.11% during the reported training process. The next chapter discusses the results and performance of the developed

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the results obtained from the implementation of the AI-Based Plant Disease Detection System using Deep Learning and Transfer Learning.

The developed system uses the MobileNetV2 architecture to classify plant leaf images into 38 supported categories. The trained model is integrated with a Flask web application that allows users to upload plant leaf images and view the prediction results.

The performance of the model is discussed using training and validation accuracy. The working of the web application is also explained through its main pages and prediction process.

5.2 Model Training Results

The model is trained using the PlantVillage dataset with the MobileNetV2 architecture. During training, the model learns visual patterns from labelled plant leaf images.

The training process is performed for 10 epochs with a batch size of 32 and an input image size of 224 × 224 pixels.

The model performance is monitored using training and validation accuracy.

5.2.1 Accuracy Results

The following accuracy values were recorded during model development.

Evaluation Metric	Accuracy
Training Accuracy	92.92%
Validation Accuracy	94.11%

The model achieved a training accuracy of 92.92% and a validation accuracy of 94.11%.

These results indicate that the model learned useful visual features from the training images and achieved the reported performance on the validation data.

Validation accuracy is slightly higher than training accuracy. This can occur due to differences in the training and validation data or the effect of training-time augmentation and other training settings.

These values represent training and validation performance. They should not be considered independent test accuracy.

5.3 Performance Analysis

The performance of the developed model is evaluated based on its ability to classify plant leaf images into the supported categories.

MobileNetV2 is used as the feature extraction architecture. Transfer learning helps reuse visual features learned by the pretrained network.

The model receives an input image of size 224 × 224 pixels and generates prediction scores for the supported classes. The class with the highest prediction score is selected as the predicted category.

The reported validation accuracy of 94.11% indicates promising performance on the validation dataset used during development.

However, prediction performance may vary when images are captured in different environments. Lighting conditions, image quality, leaf orientation, background objects, and disease severity may influence the model's output.

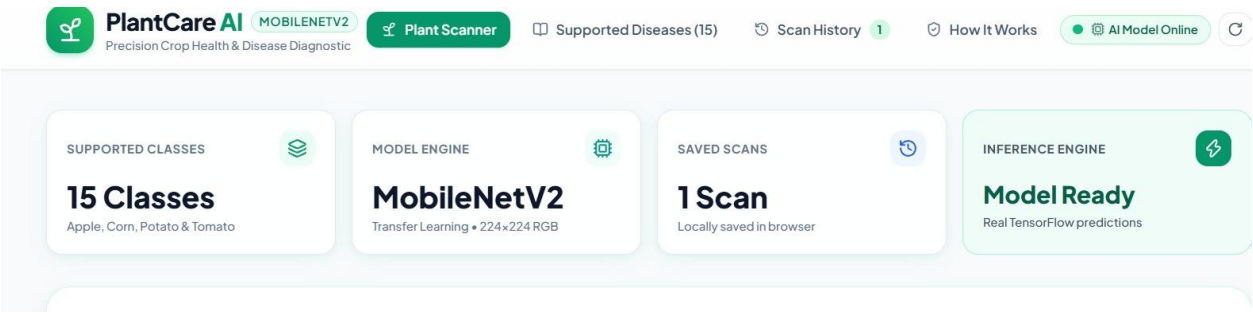
5.4 Web Application Results

The trained model is integrated with a Flask-based web application. The application provides a simple interface for uploading plant leaf images and displaying prediction results.

The web application contains the following main pages.

5.4.1 Home Page

The home page introduces the plant disease detection application. It provides access to the image scanning functionality.

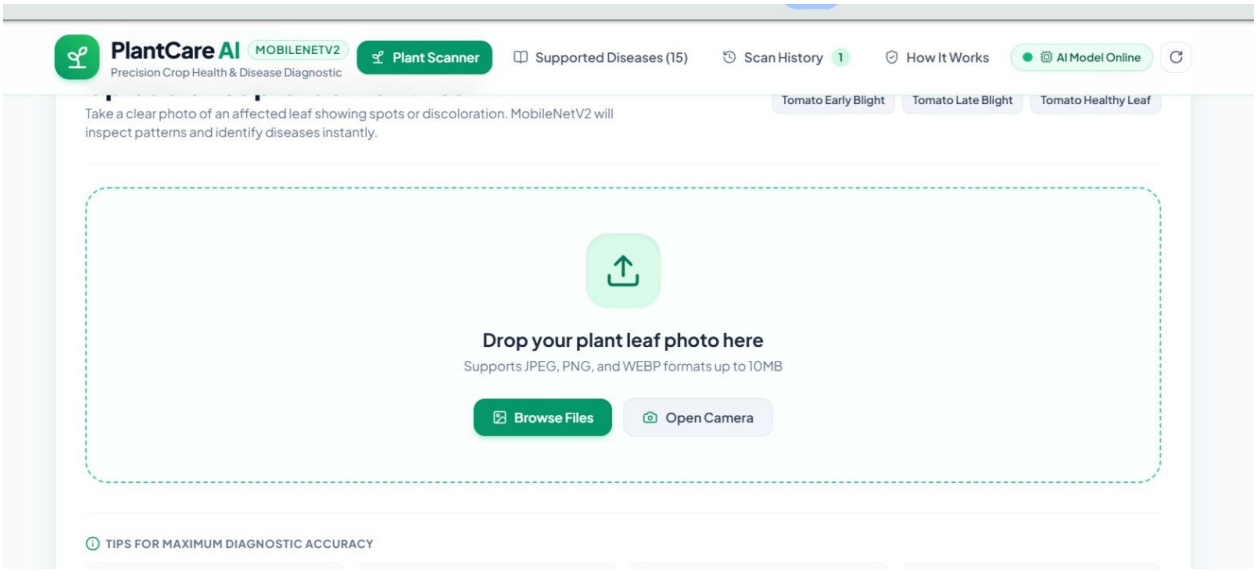


Insert Screenshot: Figure 5.1 – Home Page of Plant Disease Detection System

5.4.2 Image Scanning Page

The scanning page allows users to select and upload a plant leaf image from their device.

After the image is submitted, the Flask application receives the file and prepares it for model prediction.

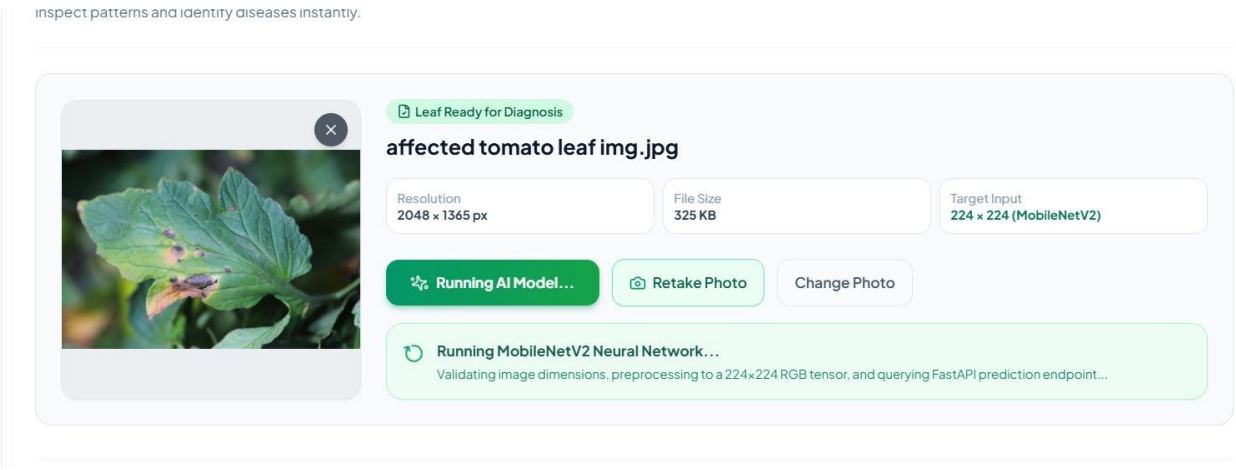


Insert Screenshot: Figure 5.2 – Plant Leaf Image Upload Page

5.4.3 Prediction Result Page

After processing the uploaded image, the trained model generates prediction scores for the supported categories.

The application identifies the predicted class and displays the category name and confidence score on the result page.



Insert Screenshot: Figure 5.3 – Plant Disease Prediction Result Page

5.5 Prediction Process Analysis

The prediction process is performed through the following stages:

1. The user selects a plant leaf image.
2. The Flask application receives the uploaded image.
3. The image is resized to 224 × 224 pixels.
4. The required preprocessing is applied.
5. The trained MobileNetV2-based model processes the image.
6. The model generates prediction scores for the supported classes.
7. The predicted class is mapped to its corresponding class name.
8. The result and confidence score are displayed on the web page.

This process connects the deep learning model with the web application and allows users to obtain classification results through a browser.

5.6 Functional Testing Results

Functional testing is performed to verify the major operations of the web application.

RESULTS AND DISCUSSION

Test Case	Expected Result	Actual Status
Home Page Loading	Home page should open	To be updated
Image Upload	Supported image should upload	To be updated
Model Prediction	Model should generate prediction scores	To be updated
Result Display	Predicted class and confidence should appear	To be updated

The actual status should be updated to **Pass** or **Fail** after performing each test in the running application.

5.7 Advantages of the Developed System

The developed system provides the following advantages:

1. It automates the classification of plant leaf images.
2. It uses MobileNetV2 transfer learning for image classification.
3. It supports 38 plant disease and healthy categories.
4. It provides a simple web-based image upload interface.
5. It displays the predicted category and confidence score.
6. It uses a saved model for prediction without repeating model training.
7. It combines deep learning technology with a Flask web application.

5.8 Limitations of the Developed System

Although the system provides automated image classification, it has certain limitations.

1. The model supports only the 38 categories included in the selected dataset.
2. Prediction performance may vary with image quality and lighting conditions.
3. Images captured in real agricultural environments may differ from dataset images.
4. The system depends on the trained model and its supported classes.
5. The predicted result is not a substitute for professional agricultural diagnosis.
6. The reported validation accuracy does not establish performance on all real-world images.

5.9 Discussion

The developed project demonstrates how deep learning and transfer learning can be applied to plant leaf image classification.

The use of MobileNetV2 provides a pretrained feature extraction architecture that can be adapted to the selected plant disease dataset. TensorFlow and Keras support model development and training, while Flask provides a connection between the trained model and the web interface.

The model achieved 92.92% training accuracy and 94.11% validation accuracy during the reported training process. The web application allows users to upload leaf images and view the model's predicted category and confidence score.

The results demonstrate the successful integration of image classification and web application development in a prototype plant disease detection system.

Further evaluation using an independent test dataset and images collected from real agricultural environments would help assess the system's generalisation ability.

5.10 Summary

This chapter presented the results and discussion of the developed plant disease detection system.

The MobileNetV2-based model achieved a training accuracy of 92.92% and validation accuracy of 94.11%. The trained model was integrated with a Flask web application to support image uploading and prediction result display.

The advantages, limitations, and functional testing approach were also discussed.

The next chapter presents the conclusion and possible future enhancements of the project.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Introduction

This chapter presents the conclusion of the project titled **"AI-Based Plant Disease Detection Using Deep Learning and Transfer Learning."** It summarises the development process, technologies used, results obtained, and possible future enhancements.

The main objective of this project is to develop an automated plant disease classification system using deep learning techniques. The system uses MobileNetV2 transfer learning to identify healthy and diseased plant leaf categories from uploaded images.

6.2 Conclusion

The project successfully demonstrates the application of Artificial Intelligence and Deep Learning in plant disease image classification.

The system is developed using Python, TensorFlow, Keras, and the MobileNetV2 architecture. The PlantVillage dataset is used to train and validate the model, which supports 38 different plant leaf categories.

Transfer learning is applied to adapt the pretrained MobileNetV2 architecture to the plant disease classification task. The images are resized to 224 × 224 pixels and processed before being supplied to the model.

The trained model achieved a training accuracy of **92.92%** and a validation accuracy of **94.11%** during the reported training process.

The trained model is integrated with a Flask web application. Users can upload plant leaf images through the web interface and view the predicted category along with its confidence score.

Overall, the project demonstrates how deep learning and web technologies can be combined to develop a simple prototype for automated plant disease classification. The system can serve as a supporting tool for preliminary identification, but its predictions should be verified by agricultural experts when making important decisions.

6.3 Project Achievements

The major achievements of the project are:

1. Developed a plant disease classification model using MobileNetV2.
2. Used the Plant Village dataset containing 38 supported classes.
3. Applied image preprocessing and data augmentation techniques.
4. Implemented transfer learning using TensorFlow and Keras.
5. Achieved 92.92% training accuracy and 94.11% validation accuracy.
6. Saved the trained model in Keras format.
7. Developed a Flask-based web application.
8. Integrated the trained model with the web interface.
9. Implemented image uploading and prediction result display.

6.4 Limitations

Although the project achieves its main objectives, some limitations remain:

1. The system supports only 38 categories included in the selected dataset.
2. Prediction results may vary depending on image quality and lighting conditions.
3. The model may perform differently on images captured in real agricultural environments.
4. The system does not guarantee the correct identification of every plant disease.
5. The reported training and validation accuracy values do not represent independent test accuracy.
6. The application is currently developed as a local Flask-based prototype.

6.5 Future Enhancements

The project can be improved in several ways to make it more useful and reliable.

6.5.1 Mobile Application Development

The existing web-based system can be extended into an Android or iOS mobile application. This would allow users to capture or upload plant leaf images directly from their mobile devices.

6.5.2 Improved Dataset

Additional plant leaf images can be collected from real agricultural environments. Images with different lighting conditions, backgrounds, and disease stages can help improve the model's ability to handle real-world situations.

6.5.3 Model Performance Improvement

The model can be improved by experimenting with different deep learning architectures, fine-tuning strategies, and training parameters. Independent test datasets can also be used for more reliable performance evaluation.

6.5.4 Disease Information Module

A future version can include a disease information section that explains the identified disease, its common symptoms, and general prevention methods. Any agricultural recommendations should be reviewed by qualified experts.

6.5.5 Support for More Plant Categories

The dataset can be expanded to include additional plant species and disease categories. This would increase the range of plants supported by the application.

6.5.6 Cloud-Based Deployment

The Flask application can be deployed on a cloud platform so that users can access the system through the internet instead of running it locally.

6.6 Final Conclusion

The project demonstrates the use of Artificial Intelligence, Deep Learning, and Transfer Learning for plant disease image classification.

By combining MobileNetV2, TensorFlow/Keras, and Flask, the developed system provides a simple method for uploading plant leaf images and obtaining classification results.

The project provides practical experience in dataset preparation, image preprocessing, model training, transfer learning, and web application development.

In conclusion, the developed system serves as a prototype for AI-based plant disease detection and provides a foundation for future improvements in automated agricultural image classification.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using Deep Learning for Image-Based Plant Disease Detection," *Frontiers in Plant Science*, vol. 7, 2016.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[3] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv:1704.04861, 2017.

[4] D. P. Hughes and M. Salathé, "An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics," arXiv preprint arXiv:1511.08060, 2015.

[5] TensorFlow, "TensorFlow Documentation." Available: https://www.tensorflow.org/

[6] Keras, "Keras API Documentation." Available: https://keras.io/

[7] Flask, "Flask Documentation." Available: https://flask.palletsprojects.com/

[8] Python Software Foundation, "Python Documentation." Available: https://docs.python.org/3/